

Documentação Técnica de um Solucionador do Quebra-Cabeça Tango por Força Bruta e Backtracking

Caio Victor Kodato Teixeira, Matheus Fernandes, Miguel Vieira de Souza
Victor Gabriel Cruz Pereira, Vitor Hugo Dutra Marinho

¹Pontifícia Universidade Católica de Minas Gerais (PUC Minas)
Curso de Engenharia de Software – Fundamentos de Projeto e Análise de Algoritmos
Belo Horizonte – MG – Brasil

Abstract. *This document presents the modeling, implementation, and analysis of a Java program designed to solve instances of the Tango puzzle. The system represents the board as a square matrix of values, stores equality and opposition constraints between adjacent cells, and provides two solving strategies: brute force and backtracking. The brute-force approach generates complete combinations and validates only the leaf nodes of the search tree. In contrast, the backtracking approach incrementally constructs the solution and prunes search branches whenever a partial game rule is violated. Experimental results show that both approaches produce the same solution for the 4x4 sample board; however, backtracking explores only 37 states compared to 21,915 states explored by brute force, highlighting the practical impact of pruning.*

Resumo. *Este documento apresenta a modelagem, a implementação e a análise de um programa em Java para resolver instâncias do quebra-cabeça Tango. O sistema representa o tabuleiro como uma matriz quadrada de valores, armazena restrições de igualdade e oposição entre células vizinhas e disponibiliza duas estratégias de resolução: força bruta e backtracking. A força bruta gera combinações completas e valida apenas as folhas da busca. O backtracking constrói a solução de forma incremental e aplica podas sempre que uma regra parcial do jogo já foi violada. Os testes realizados mostram a mesma solução para o tabuleiro 4x4 de exemplo, com 37 estados visitados no backtracking contra 21.915 na força bruta, evidenciando o impacto prático das podas.*

Trabalho Prático 2 – FPAA

Docente: Prof. João Pedro O. Batisteli
Tecnologia: Java, execução via console.

1. Introdução

Problemas de satisfação de restrições aparecem quando o objetivo não é apenas produzir uma configuração qualquer, mas encontrar um estado que obedeça simultaneamente a um conjunto de regras. O quebra-cabeça Tango se encaixa diretamente nesse modelo: cada célula vazia deve receber um de dois símbolos, Sol ou Lua, e a solução final precisa respeitar limites de adjacência, equilíbrio entre linhas e colunas e relações locais de igualdade ou oposição.

O objetivo do projeto foi construir um solucionador capaz de ler um tabuleiro inicial, aplicar as regras do Tango e imprimir no terminal a configuração original e a configuração resolvida. Para evidenciar a diferença entre busca exaustiva e busca com podas, foram implementadas duas estratégias: uma força bruta, que valida apenas combinações completas, e um backtracking, que interrompe ramos inválidos assim que alguma restrição parcial é detectada.

2. Modelagem do Problema

2.1. Representação do tabuleiro

O tabuleiro foi modelado pela classe `Board`. Cada instância possui um tamanho par `size`, uma matriz mutável `CellValue[][] cells` e uma lista de restrições. O construtor rejeita tamanhos não positivos ou ímpares, pois a regra de equilíbrio exige que cada linha e coluna tenha exatamente metade dos símbolos de cada tipo.

As células são representadas pelo enum `CellValue`, com os valores `VAZIO`, `SOL` e `LUA`. O valor `VAZIO` não corresponde a um terceiro símbolo do jogo; ele indica apenas que a decisão ainda não foi tomada. Na leitura e na impressão, os caracteres utilizados são `.`, `S` e `L`.

2.2. Representação das restrições

Cada restrição é representada pela classe imutável `Constraint`, que armazena duas coordenadas e um tipo de relação. A coordenada é definida por `Position`, com `row` e `column` iniciados em zero. O tipo da relação é definido pelo enum `ConstraintType`: `EQUAL`, para o símbolo `=`, e `OPPOSITE`, para o símbolo `x`. Antes da resolução, `BoardInputValidator` confirma que cada restrição liga células vizinhas, isto é, posições com distância de Manhattan igual a 1.

2.3. Formato de entrada

O formato textual escolhido prioriza simplicidade e reprodutibilidade. A entrada possui uma linha obrigatória `size=`, uma seção `board:` com exatamente `size` linhas e uma seção opcional `constraints:`. Linhas vazias e comentários iniciados por `#` são ignorados pelo leitor.

```
size=4
board:
S...
....
..L.
....
constraints:
0 0 0 1 x
2 2 3 2 =
```

Listing 1. Exemplo de arquivo de entrada 4x4.

O pacote `tango.io` concentra a leitura, validação preliminar e impressão. A classe `InputReader` converte o arquivo em objeto `Board`; `BoardInputValidator` rejeita violações já determinadas antes da busca; e `BoardPrinter` exibe a grade com índices de linha e coluna, além da lista de restrições.

3. Modularização da Implementação

A organização do código separa o modelo do problema, as regras do jogo, os algoritmos e a interface de execução. Essa separação evita que a recursão dos solvers fique misturada com a validação das regras, requisito importante para comparar corretamente força bruta e backtracking.

Tabela 1. Pacotes principais do projeto.

Pacote	Responsabilidade
tango.model	Estruturas centrais: tabuleiro, células, posições e restrições.
tango.validation	Regras finais e parciais do Tango, usadas como validação e poda.
tango.solver	Contrato comum Solver, resultado da busca e implementações de força bruta e backtracking.
tango.io	Leitura de arquivos, validação de entrada, catálogo, geração e impressão do tabuleiro.
tango.app	Fachada da aplicação, menu interativo e integração entre entrada, solver e saída.

O validador principal é `BoardValidator`. O método `isFinalValid` verifica uma solução completa, enquanto `isPartialValid` verifica apenas violações que já não podem ser corrigidas. Essa distinção é a base da diferença entre os dois algoritmos.

4. Estratégia de Resolução por Força Bruta

A estratégia de força bruta está implementada em `BruteForceSolver`. Primeiro, o algoritmo copia o tabuleiro inicial para preservar as dicas originais e coleta todas as posições vazias. Em seguida, executa uma recursão que, para cada posição vazia, testa os valores SOL e LUA. Nenhuma regra parcial é usada como poda durante esse processo.

A condição de parada ocorre quando todas as posições vazias já receberam um valor. Somente nesse momento o algoritmo chama `validator.isFinalValid(board)`. Se o tabuleiro completo satisfaz as cinco regras, a primeira solução encontrada é retornada. Caso contrário, a recursão retrocede e continua a enumeração das demais combinações.

```
forcaBruta(tabuleiro, posicoesVazias, i):
    se i == quantidade de posicoes vazias:
        retornar validacaoFinal(tabuleiro)

    posicao = posicoesVazias[i]
    para valor em {SOL, LUA}:
        tabuleiro[posicao] = valor
        estadosVisitados++
        se forcaBruta(tabuleiro, posicoesVazias, i + 1):
            retornar verdadeiro

    tabuleiro[posicao] = VAZIO
    retornar falso
```

Listing 2. Pseudocódigo resumido da força bruta.

Essa abordagem corresponde à geração do espaço completo de estados possíveis para as células vazias. Se houver k células vazias, existem até 2^k configurações completas. O programa interrompe a busca ao encontrar a primeira solução válida, mas o comportamento geral continua sendo exponencial.

5. Estratégia de Resolução por Backtracking

O backtracking está implementado em `BacktrackingSolver`. A estrutura recursiva é semelhante à da força bruta: o algoritmo copia o tabuleiro, lista as posições vazias e tenta preencher cada uma com SOL ou LUA. A diferença central está no uso de `validator.isPartialValid(board)` após cada atribuição candidata.

A validação parcial aplica as regras que já podem invalidar definitivamente um ramo da busca. O algoritmo rejeita imediatamente estados com três símbolos iguais consecutivos, linhas ou colunas com quantidade de SOL ou LUA superior a `size / 2`, ou restrições de igualdade/oposição já violadas por duas células preenchidas. Restrições com alguma célula ainda vazia não reprovam o estado, pois ainda podem ser satisfeitas por escolhas futuras.

```
backtracking(tabuleiro, posicoesVazias, i):
    se i == quantidade de posicoes vazias:
        retornar validacaoFinal(tabuleiro)

    posicao = posicoesVazias[i]
    para valor em {SOL, LUA}:
        tabuleiro[posicao] = valor
        estadosVisitados++
        se validacaoParcial(tabuleiro) e backtracking(tabuleiro,
posicoesVazias, i + 1):
            retornar verdadeiro

    tabuleiro[posicao] = VAZIO
    retornar falso
```

Listing 3. Pseudocódigo resumido do backtracking.

A condição base continua sendo o preenchimento de todas as células originalmente vazias. Entretanto, muitos ramos deixam de chegar às folhas porque são cortados antes. Essa poda não altera a corretude: ela elimina apenas estados que já desrespeitam alguma regra obrigatória do jogo.

6. Regras de Validação e Poda

As cinco regras do Tango aparecem em métodos separados para facilitar leitura, teste e reutilização. A regra de preenchimento completo é aplicada somente na validação final. As regras de adjacência, equilíbrio e restrições são usadas tanto no final quanto parcialmente, com critérios adaptados para tabuleiros incompletos.

Tabela 2. Relação entre regras do problema e métodos de validação.

Regra	Método	Uso no algoritmo
Preenchimento completo	hasNoEmptyCells hasNoThreeEqual	Aplicado na validação final.
Máximo de dois iguais adjacentes	AdjacentCells	Aplicado como validação final e poda parcial.
Equilíbrio de linhas e colunas	hasValidRowAnd ColumnBalance / hasValidPartialRowAnd ColumnBalance	No final exige contagem exata; na poda rejeita apenas excesso.
Igualdade =	hasValidConstraints	Compara quando as duas células da relação estão preenchidas.
Oposição x	hasValidConstraints	Rejeita duas células iguais em restrição de oposição.

Essa separação também atende à exigência de modularização: a mecânica de recursão está nos solvers, enquanto as regras de aceitação ou rejeição ficam no pacote `tango.validation`.

7. Exemplos de Execução

O projeto foi compilado diretamente pelo terminal, sem dependências externas. O comando utilizado para compilar foi:

```
javac -d out $(find src/main/java -name "*.java")
```

Em seguida, foram executados testes com tabuleiros distintos e com os dois algoritmos. A Tabela 3 resume os resultados observados.

Tabela 3. Resumo dos testes executados.

Instância	Entrada	Algoritmo	Estados
4x4 com duas restrições	exemplo 4x4	Backtracking	37
4x4 com duas restrições	exemplo 4x4	Força bruta	21.915
4x4 gerado automaticamente	automático 001	Backtracking	17
8x8 gerado automaticamente	automático 002	Backtracking	2.315
Entrada inválida com SSS na primeira linha	teste inválido	Backtracking	Rejeitada

No tabuleiro 4x4 de exemplo, os dois algoritmos retornaram a mesma solução, mas com diferença expressiva no número de estados visitados. O recorte a seguir mostra o caso com backtracking:

```
java -cp out tango.Main examples/tabuleiro-exemplo-4x4.txt  
backtracking  
  
Tabuleiro resolvido:  
  0 1 2 3  
0  S L S L  
1  S L S L  
2  L S L S  
3  L S L S  
Restricoes:
```

```
(0, 0) x (0, 1)
(2, 2) = (3, 2)

Estados visitados: 37
```

Listing 4. Recorte da execução do 4x4 com backtracking.

A execução da mesma entrada com força bruta encontrou a mesma grade final, porém visitou 21.915 estados. Esse contraste demonstra que a diferença entre os algoritmos não está na solução obtida, mas na quantidade de ramos explorados até alcançá-la. O recorte a seguir apresenta essa execução:

```
java -cp out tango.Main examples/tabuleiro-exemplo-4x4.txt forca-
bruta

Tabuleiro resolvido:
  0 1 2 3
0  S L S L
1  S L S L
2  L S L S
3  L S L S
Restricoes:
  (0, 0) x (0, 1)
  (2, 2) = (3, 2)

Estados visitados: 21915
```

Listing 5. Recorte da execução do 4x4 com força bruta.

Os tabuleiros gerados automaticamente pela própria interface também foram resolvidos. O primeiro é um 4x4 com quatro restrições, resolvido pelo backtracking em apenas 17 estados:

```
java -cp out tango.Main examples/tabuleiros/automatico
-001-20260620-220231.txt backtracking

Tabuleiro inicial:
  0 1 2 3
0  L . S .
1  . S . L
2  S . L .
3  . . . .
Restricoes:
  (2, 1) x (3, 1)
  (2, 0) = (2, 1)
  (1, 2) x (1, 3)
  (0, 0) = (0, 1)

Tabuleiro resolvido:
  0 1 2 3
0  L L S S
1  L S S L
2  S S L L
3  S L L S

Estados visitados: 17
```

Listing 6. Recorte da execução do 4x4 gerado automaticamente (automático 001).

Para demonstrar o comportamento em uma instância maior, o recorte a seguir mostra um tabuleiro 8x8 gerado automaticamente (automático 002), com a configuração inicial e a final produzidas pelo backtracking em 2.315 estados:

```
java -cp out tango.Main examples/tabuleiros/automatico
-002-20260620-220418.txt backtracking
```

Tabuleiro inicial:

	0	1	2	3	4	5	6	7
0	.	.	.	S	.	S	.	.
1	.	.	.	.	S	.	L	.
2	.	S	.	.	.	S	.	.
3	.	.	.	.	.	L	S	.
4	.	S	L	.	.	.	.	.
5	S	L	.	.	.	L	.	.
6	.	.	.	L	.	S	.	.
7	.	L	.	.	.	.	.	.

Tabuleiro resolvido:

	0	1	2	3	4	5	6	7
0	L	S	L	S	L	S	S	L
1	S	L	S	L	S	L	L	S
2	L	S	L	S	L	S	S	L
3	L	L	S	L	S	L	S	S
4	S	S	L	S	L	S	L	L
5	S	L	S	L	S	L	S	L
6	L	S	S	L	L	S	L	S
7	S	L	L	S	S	L	L	S

Estados visitados: 2315

Listing 7. Recorte da execução do 8x8 gerado automaticamente (automático 002).

Por fim, o sistema rejeita entradas inválidas antes mesmo de iniciar a busca, informando a regra violada:

```
Erro: o tabuleiro possui tres simbolos iguais consecutivos em uma
      linha ou coluna.
```

TP2 FPAA - Tango Solver

Modo por comando:

```
java -cp out tango.Main <arquivo-entrada> [algoritmo]
```

Listing 8. Recorte da execução de uma entrada inválida.

8. Análise de Complexidade

Considere n como a dimensão do tabuleiro, k como a quantidade de células vazias no estado inicial e r como a quantidade de restrições. Como cada posição vazia possui duas possibilidades, o espaço bruto de busca possui até 2^k folhas.

Na força bruta, a recursão pode gerar a árvore binária de decisões até encontrar uma solução. No pior caso, a busca visita ordem exponencial de estados. Cada folha

completa é verificada por `isFinalValid`, cuja varredura custa $O(n^2 + r)$, pois percorre células da grade, linhas, colunas e lista de restrições. Portanto, a ordem teórica de pior caso pode ser descrita como:

$$T_{FB}(n, k, r) = O(2^k \cdot (n^2 + r)).$$

No backtracking, o pior caso assintótico permanece exponencial, pois pode existir uma instância em que as podas não eliminem ramos cedo. Ainda assim, a validação parcial reduz o espaço explorado em instâncias reais, porque ramos com trios inválidos, excesso de símbolos em linhas ou colunas e restrições já violadas são interrompidos antes de se tornarem tabuleiros completos. A complexidade de pior caso pode ser escrita como:

$$T_{BT}(n, k, r) = O(b \cdot (n^2 + r)), \quad b \leq 2^k,$$

em que b representa a quantidade efetiva de estados que sobreviveram às podas. O ganho prático aparece quando b fica muito menor que 2^k , como ocorreu no 4x4 testado: 37 estados no backtracking contra 21.915 na força bruta.

Quanto ao espaço, os dois algoritmos mantêm uma cópia do tabuleiro, a lista de posições vazias e a pilha de recursão. Assim, o custo espacial é $O(n^2 + k)$, desconsiderando estruturas auxiliares constantes do validador.

9. Conclusão

O projeto implementa corretamente um solucionador de Tango em Java, operando pelo console e sem uso de bibliotecas de CSP ou solvers prontos. A modelagem separa o estado do tabuleiro, os valores das células, as restrições e as posições. A implementação também separa a validação das regras da mecânica recursiva dos algoritmos, permitindo comparar a busca exaustiva com o backtracking.

A força bruta cumpre o papel de gerar combinações completas e validar apenas as soluções finais. O backtracking preserva a mesma ideia, mas acrescenta podas baseadas nas cinco regras do jogo. Os testes confirmam que ambas as estratégias encontram soluções válidas, enquanto o backtracking reduz significativamente o número de estados visitados nas instâncias avaliadas. Dessa forma, o sistema atende aos requisitos centrais do trabalho: leitura e representação do tabuleiro, execução em terminal, impressão do estado inicial e final, implementação de força bruta e backtracking, modularização e análise do espaço de busca.